LI.FI

CATValidatorV2 (v2.0.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1 About Researcher **1**

2 Disclaimer **1**

3 Scope **1**

4 Risk classification **1**
- 4.1 Impact . . . 1
- 4.2 Likelihood . . . 2
- 4.3 Action required for severity levels . . . 2

5 Executive Summary **2**

6 Findings **3**
- 6.1 Low Risk . . . 3
  - 6.1.1 Existing validator balances can break exact `paid[]` checks . . . 3
  - 6.1.2 Capped and sender-fee tokens can make outcome forwarding revert . . . 4
- 6.2 Informational . . . 5
  - 6.2.1 The default artifact uses `MCOPY` despite the pre-Cancun compatibility claim . . . 5
  - 6.2.2 Nonce state is local to each validator . . . 6
  - 6.2.3 Same-token flow checks can pass without executing the intended action . . . 6
  - 6.2.4 The SDK refund helper uses fixed allowance amounts . . . 7
  - 6.2.5 `spent[]` records the transfer amount instead of the escrow's actual debit . . . 7
  - 6.2.6 Outcome destination behavior can block settlement or change validation inputs . . . 8

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ethereum, Ripple, Uniswap, Morpho, Injective, Phantom, Cosmos and received bug-bounties from those projects.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

CATValidatorV2	`solidity/src/CATValidatorV2.sol (v2.0.0)`
LibExecutionConstraintV2	`solidity/src/libs/LibExecutionConstraintV2.sol`
LibValidationVM	`solidity/src/libs/LibValidationVM.sol`

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 20 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **8 issues** were found.

Project Summary

Project Name	**CATValidatorV2 v2.0.0**
Project Type	Solidity Smart Contracts
Repository	lifinance/catapultar
Commit	c8709fcab
Audit Timeline	October 18, 2026 - October 20, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	**0**
High Risk	**0**
Medium Risk	**0**
Low Risk	**2**
Gas Optimizations	**0**
Informational	**6**
Total Issues	**8**

6 Findings

6.1 Low Risk

6.1.1 Existing validator balances can break exact `paid[]` checks

CONTEXT: CATValidator.sol#L184-L187

DESCRIPTION: The validator forwards its complete balance for each outcome token. V2 records the forwarded amount in `paid[]`, so tokens or native value already held by the validator are counted as part of the fill. Anyone can change that value by transferring funds to the validator before settlement.

An exact, maximum, or bounded payment check can then receive `x + 1` when the fill delivered `x`. If the program rejects that value, the settlement reverts and restores the donated balance. The same unit remains in place for the next attempt. A committed fill may be unable to compensate by delivering one unit less.

Minimum checks such as `paid >= x` are unaffected because the surplus goes to the signed destination. Failed settlements also leave the escrow untouched and preserve the refund path.

POC: This case is in `solidity/test/audit/FlowRegisterSemantics.t.sol`. `ExactRegisterVM` accepts a payment of exactly 100. Register 0 contains the account, so `paid[0]` is register 1 when there are no params or allowances.

```
contract ExactRegisterVM {
    error UnexpectedRegister(uint256 actual, uint256 expected);

    uint256 immutable REGISTER;
    uint256 immutable EXPECTED;

    constructor(uint256 register_, uint256 expected_) {
        REGISTER = register_;
        EXPECTED = expected_;
    }

    function runVM(VMCommand[] calldata, VMState calldata state) external view {
        uint256 actual = abi.decode(state.registers[REGISTER], (uint256));
        if (actual != EXPECTED) revert UnexpectedRegister(actual, EXPECTED);
    }
}

function test_ambientBalanceChangesPaidRegisterAndPersistsOnRevert() external {
    uint256 expected = 100 ether;
    CATValidatorV2 target = new CATValidatorV2(address(new ExactRegisterVM(1, expected)));
    Outcome[] memory outcomes = oneOutcome(address(token), expected, destination);
    VMCommand[] memory program = dummyProgram();
    bytes memory payload = abi.encodeCall(OutcomeFill.mint, (token, address(target), expected));
    bytes memory signature = sign(
        target, 1, noAllowances(), outcomes, hashProgram(program), bytes32(0)
    );

    token.mint(address(target), 1);

    vm.expectRevert(
        abi.encodeWithSelector(
            CATValidatorV2.ValidationFailed.selector,
            abi.encodeWithSelector(ExactRegisterVM.UnexpectedRegister.selector, expected + 1, expected)
        )
    );
    vm.prank(executor);
    target.entry(
        address(fill), payload, signer, 1, noAllowances(), outcomes,
        unbound(), program, noParams(), signature
    );

    assertEq(token.balanceOf(address(target)), 1);
    assertEq(token.balanceOf(destination), 0);
    assertFalse(target.spentNonces(signer, 1));
}
```

RECOMMENDATION: Generate only lower-bound checks over `paid[]` and document that the register includes balances already held by the validator. Exact or upper-bound checks require a new register that measures the amount delivered by the current fill.

LI.FI: Acknowledged. paid[j] is the gross amount forwarded and includes any balance the validator held before the fill. 1652038 documents this in the V2 NatSpec and the README, with the rule that only lower-bound (>=) checks over paid are safe. Our program producer emits lower-bound checks only.

RESEARCHER: Acknowledged.

6.1.2 Capped and sender-fee tokens can make outcome forwarding revert

CONTEXT: CATValidator.sol#L184-L187

DESCRIPTION: `_validatePayment` sends the validator's complete token balance in one transfer. A capped token reverts when that balance exceeds its per-transfer limit. A token that charges an extra fee to the sender also reverts: after the complete balance has moved, the validator has nothing left to pay the fee.

Capped tokens introduce a griefing case because anyone can add tokens to the validator until its balance is above the cap. Every settlement then retries the same oversized transfer, and there is no partial-sweep path.

The revert rolls back the settlement, including the nonce and any escrow movement. The owner can still use the signed refund, but the execute constraint cannot settle with the affected token. This behavior is inherited from V1 and does not affect standard ERC-20 transfers.

POC:

The test cases below are in `solidity/test/audit/LivenessAndSemantics.t.sol`. The shared fixture deploys the validator and signs the displayed outcomes for nonce 1.

```
contract CappedToken is MockERC20 {
    uint256 immutable cap;

    constructor(uint256 cap_) MockERC20("Capped", "CAP", 18) {
        cap = cap_;
    }

    function transfer(address to, uint256 amount) public override returns (bool) {
        require(amount <= cap, "cap exceeded");
        return super.transfer(to, amount);
    }
}

contract SenderFeeOutcomeToken is MockERC20 {
    constructor() MockERC20("Sender fee", "FEE", 18) { }

    function transfer(address to, uint256 amount) public override returns (bool) {
        _transfer(msg.sender, to, amount);
        _burn(msg.sender, amount / 10);
        return true;
    }
}

function test_cappedTokenOutcome_bricksSettlement() external {
    capped.mint(address(validator), 1000 ether);
    Outcome[] memory outcomes = new Outcome[](1);
    outcomes[0] = Outcome(address(capped), 900 ether, destination);
    bytes memory signature = sign(new AllowanceSpend[](0), outcomes);

    vm.prank(executor);
    vm.expectRevert();
    validator.entry(
        address(0), hex"", signer, 1, new AllowanceSpend[](0), outcomes,
        ExecutionBinding(address(0), bytes32(0)),
        new VMCommand[](0), new bytes32[](0), signature
    );
}
```

```
function test_senderFeeOutcome_bricksSettlement() external {
    SenderFeeOutcomeToken feeToken = new SenderFeeOutcomeToken();
    feeToken.mint(address(validator), 100 ether);
    Outcome[] memory outcomes = new Outcome[](1);
    outcomes[0] = Outcome(address(feeToken), 100 ether, destination);
    bytes memory signature = sign(new AllowanceSpend[](0), outcomes);

    vm.prank(executor);
    vm.expectRevert();
    validator.entry(
        address(0), hex"", signer, 1, new AllowanceSpend[](0), outcomes,
        ExecutionBinding(address(0), bytes32(0)),
        new VMCommand[](0), new bytes32[](0), signature
    );

    assertEq(feeToken.balanceOf(address(validator)), 100 ether);
    assertEq(feeToken.balanceOf(destination), 0);
    assertFalse(validator.spentNonces(signer, 1));
}
```

RECOMMENDATION: Allow outcome tokens only if they can transfer the validator's complete balance and do not charge an additional sender fee. Document this requirement in the producer and enforce it in any token allowlist.

LI.FI: Acknowledged. Forwarding the complete balance in one transfer is inherited from v1. 1652038 documents the outcome-token requirements in the V2 NatSpec and the README: the token must allow a single transfer of the validator's complete balance and must not charge the sender a fee on top of the amount, and a per-transfer cap does not meet this because anyone can push the validator's balance over it. A failed push reverts the settlement, so the escrow keeps its funds, the nonce stays unspent and the refund constraint is the recovery route. Standard fee-on-transfer tokens, which deduct the fee from the amount, are unaffected, and our composer sizes outcome floors for them. No contract change planned.

RESEARCHER: Acknowledged.

6.2 Informational

6.2.1 The default artifact uses `MCOPY` despite the pre-Cancun compatibility claim

CONTEXT: foundry.toml#L1-L9

DESCRIPTION: The default Foundry profile does not set `evm_version`. Foundry 1.7.1 and solc 0.8.35 resolve that profile to Osaka, and the compiler inserts the Cancun `MCOPY` opcode into the deployed V2 runtime.

Tstorish selects SSTORE/SLOAD fallbacks when transient storage is unavailable, but that choice covers only its own storage operations. A Shanghai VM can deploy the artifact, then fails when a non-empty validation program reaches the compiler-generated `MCOPY`. Empty-program constraints avoid `_runValidation`, so the bound refund path remains available.

Compiling the repository for Shanghai also fails because solc rejects Tstorish's explicit `tload` and `tstore` instructions for that target. Supporting pre-Cancun chains requires separate handling for the transient storage opcodes.

RECOMMENDATION: If V2 requires Cancun, pin `evm_version = "cancun"`, remove the pre-Cancun claim, and block deployments on older VMs. If Shanghai support is required, isolate the transient-storage opcodes from the main runtime and add a deployment test that executes a non-empty program on a Shanghai node.

LI.FI: Confirmed. The v1 runtime built by the same toolchain also contains MCOPY, so the validator family already required Cancun; the fallback claim was wrong and is removed (1652038). foundry.toml now pins evm_version = ”cancun”, with the v1 bytecode unchanged against its pin (886c62a), and script/deploy.s.sol reverts ChainLacksCancun before deploying CATValidatorV2 when the target chain's node fails a TSTORE/MCOPY probe (5a59bb3).

RESEARCHER: Verified fix.

6.2.2 Nonce state is local to each validator

CONTEXT: CATValidator.sol#L106-L108

DESCRIPTION: Each validator has its own `spentNonces` storage. Spending nonce `n` in V1 leaves the same nonce unspent in V2 and in every other validator.

The validator address is the EIP-712 domain's `verifyingContract`, so using the same nonce on two validators still requires two separately approved digests. The SDK also checks nonce state against a specific validator address and does not create multi-validator approvals on its own.

The distinction still matters operationally during migration. If an owner has intentionally approved old and new validator digests for the same reusable escrow, settling one does not revoke the other. A stale approval can apply to a later deposit under its original signed terms.

RECOMMENDATION: State in migration guidance that nonces are validator-local. When moving a reusable escrow, approve the V2 digests and unset obsolete V1 digests in the same owner-authorized batch. Prefer single-use escrow addresses.

LI.FI: Acknowledged. Nonce scope is per validator by storage layout. 1652038 changes the v1 NatSpec to say approvals are invalidated "within this validator contract", and the V2 NatSpec and the README state that spending nonce n on one validator leaves it unspent on the other. Escrows are single-use, so no flow migrates an approval between validators.

RESEARCHER: Verifed added documentation.

6.2.3 Same-token flow checks can pass without executing the intended action

CONTEXT: CATValidatorV2.sol#L173-L175

DESCRIPTION: When the same token appears in the allowances and outcomes, the named executor can send the pulled allowance back to the validator. The validator then forwards those tokens to the signed destination. The outcome floor passes, and a rate check over `spent[]` and `paid[]` sees the expected flow even though no application action was performed.

Binding only the target does not help when that target executes arbitrary calls. Binding only the payload allows the executor to send those bytes to a contract that gives them different semantics. An exact call commitment requires both fields.

The executor is authorized, the allowance limit is respected, and the tokens reach the signed destination. An unbound constraint permits this behavior, so producers cannot use token flow alone as proof of work.

RECOMMENDATION: Document that flow registers prove value movement without proving an application action. If allowance and outcome tokens overlap, bind both the target and payload, check an action-specific state change, or reject the configuration.

LI.FI: Acknowledged. The V2 NatSpec and the README now state that spent and paid prove value moved, not that an application action ran, and that both executionTarget and payloadHash must be signed to commit the action (1652038; f839416 clarifies that the binding is optional). Whether to bind is the intent author's choice: an unbound constraint deliberately commits only to the token flow, and an author who needs a specific action performed signs a bound fill. The SDK now supports this (587a590): setExecutionBinding signs ExecutionBinding { executionTarget, payloadHash } into the V2 execute digest, and asExecuteCall rejects a target or payload that does not match before building the call. Our composer offers the same as exact legs, which bind the escrow to the leg's compiled calldata.

RESEARCHER: Verified documentation added.

6.2.4 The SDK refund helper uses fixed allowance amounts

CONTEXT: constrainedtransaction.ts#L419-L421

DESCRIPTION: `asRefundCall` copies each signed `allowance.amount` into both `allocated` and `spend`. The resulting refund reverts if the escrow holds less than that amount and leaves any excess balance behind.

The helper's documentation says it refunds the "full allowances," which can be read as sweeping the escrow's current balances. Callers handling lossy deposits or reused escrows need to build a `SPEND_BALANCE_OF_MAGIC` refund themselves.

RECOMMENDATION: State that `asRefundCall` attempts the signed allowance amounts, may fail when the escrow is underfunded, and does not sweep excess balances. Point amount-flexible integrations to the balance-sweep form.

LI.FI: Fixed in the SDK in 587a590. For CATValidatorV2, the refund now signs its allowances at maxUint256, runs with spend = SPEND_FULL_BALANCE and commits no outcomes, so it sweeps the escrow's whole balance whether the escrow holds less or more than the signed amounts. The V2 refund binding pins the target to the signed refund wallet, so the unbounded allowance has one destination. The v1 refund keeps the fixed amounts: the v1 digest signs no target, so the 1:1 outcome floor to the refund wallet is what pins the destination, and a balance sweep would let the executor redirect it. Its doc comment now states that it delivers the signed amounts, reverts when the escrow holds less, and leaves any excess. Tests cover underfunded and overfunded escrows on both versions and an executor redirecting the v1 refund (reverts); a pinned vector shows the v1 refund digest and escrow address are unchanged.

RESEARCHER: Verified fix.

6.2.5 `spent[]` records the transfer amount instead of the escrow's actual debit

CONTEXT: CATValidatorV2.sol#L260-L261

DESCRIPTION: For a literal allowance, `_handleAllowancesRecorded` stores `allowance.spend` and passes the same value to `transferFrom`. A token may charge an additional fee to the sender, so an argument of 100 can debit 110 from the escrow while the VM receives `spent[0] == 100`. A rate check based on `spent[]` would understate the escrow's cost for that token.

This matches V1 and the documented meaning of the register. The fix in `b0a9c2e` also records a `SPEND_BALANCE_OF_MAGIC` value at the point where the allowance loop reads it, which resolves the earlier replay divergence. Only tokens that debit more than the requested transfer remain affected.

RECOMMENDATION: Exclude sender-fee tokens from rate programs that use `spent[]`. Supporting them would require a separately measured debit or a trusted pre-settlement balance that the program can compare with the final balance.

LI.FI: Acknowledged; by design spent[i] is the amount passed to transferFrom. 1652038 documents in the V2 NatSpec and the README that it equals the escrow's debit only for tokens that debit exactly the transfer amount, so for a token that charges the sender a fee on top, a rate check over spent[i] understates what the escrow paid.

RESEARCHER: Acknowledged.

6.2.6 Outcome destination behavior can block settlement or change validation inputs

CONTEXT: CATValidator.sol#L178-L188

DESCRIPTION: `_validatePayment` pushes each outcome to its signed destination. The whole settlement reverts if a native receiver rejects the payment or if a token pauses or blocklists either party before settlement.

Contract recipients and callback-enabled tokens also run code during payment. `_runValidation` runs only after every outcome has been sent, so the VM reads any state changed by those callbacks. Programs that read external state need to account for changes made by the outcome destinations themselves.

A failed payout rolls back the execute attempt and leaves the escrow unchanged. The signer selected the destination, and the bound refund remains available.

RECOMMENDATION: Document that outcome destinations must remain payable through settlement and that validation observes state after all outcome callbacks.

LI.FI: Acknowledged. Pushing each outcome to its signer-committed destination during settlement is inherited from v1. A failed push reverts the whole settlement, so the escrow keeps its funds, the nonce stays unspent and the refund constraint remains available. 1652038 documents in the V2 NatSpec and the README that destinations must accept the payment at settlement, and that the program observes external state as destination and token callbacks left it.

RESEARCHER: Verified added documentation.